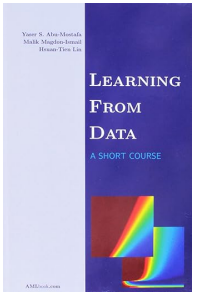
MSML610: Advanced Machine Learning

Lesson 05.3: Learn-Validation



Instructor: Dr. GP Saggese, gsaggese@umd.edu

References:

- Abu-Mostafa et al.: *"Learning From Data"* (2012)

- ***Learn-Validation Approach***
  - Train / Test
  - Cross-Validation

- Learn-Validation Approach
  - ***Train / Test***
  - Cross-Validation

Estimating Out-Of-Sample Error with One Point

- Pick a **single out-of-sample point** $(\underline{\boldsymbol{x}}', y)$
- The error of the model h is:

$$E_{val}(h) = e(h(\underline{\boldsymbol{x}}'), y)$$

  where the error can be:

  - Squared error $(h(\underline{\boldsymbol{x}}) - y)^2$
  - Binary error $I[h(\underline{\boldsymbol{x}}) - y]$
  - ...

- The **error on an out-of-sample point** is an **unbiased estimate** of E_{out}, since:

$$\mathbb{E}[E_{val}(h)] = \mathbb{E}[e(h(\underline{\boldsymbol{x}}), y)] = E_{out}$$

- The quality of the estimate depends on the standard error $\mathbb{V}[e(h(\underline{\boldsymbol{x}}), y)]$, which in an unknown value

Estimating Out-Of-Sample Error with K Points

- To improve the estimate, use a validation set with K points $(\underline{\boldsymbol{x}}_1, y_1), \ldots, (\underline{\boldsymbol{x}}_K, y_K)$ drawn IID
- Compute the **error on the validation set** as:

$$E_{val}(h) = \frac{1}{K} \sum_{i=1}^{K} e(h(\underline{\boldsymbol{x}}_i), y_i)$$

- The validation error is an **unbiased estimate** of out-of-sample error since:

$$\mathbb{E}[E_{val}(h)] = E_{out}(h)$$

- The standard error is $\sqrt{K}$ smaller:

$$\begin{aligned}
\mathbb{V}[E_{val}(h)] &= \frac{1}{K^2} \sum_i \mathbb{V}[e(h(\underline{\boldsymbol{x}}_i), y_i)] + \text{covariances} \\
&\quad \text{(covariances are 0 because } \underline{\boldsymbol{x}}_i \text{ independent)} \\
&= \frac{1}{K^2} \sum_i \mathbb{V}[e(h(\underline{\boldsymbol{x}}_i), y_i)] \\
&= \frac{K\sigma^2}{K^2} = \frac{\sigma^2}{K}
\end{aligned}$$

Trade-Off Between Training and Validation Set

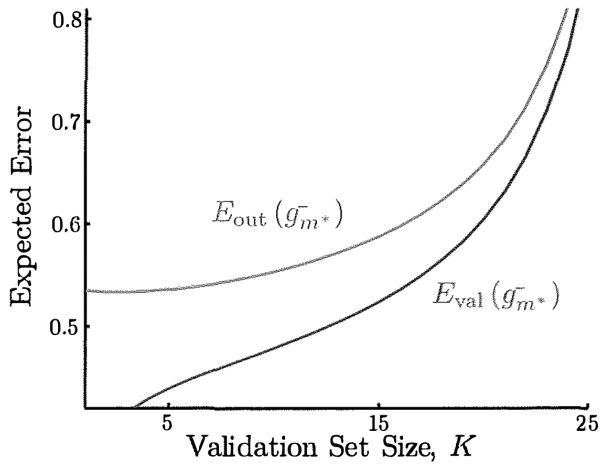
- **Problem**: you have finite amount of data points N

$$D_{val} = \{K \text{ points}\}$$
$$D_{train} = \{N - K \text{ points}\}$$

- You know that:

$$\uparrow K \implies |E_{val} - E_{out}| \downarrow \text{ (most reliable estima}$$
$$\implies \downarrow N - K \implies E_{in}, E_{out} \uparrow \text{ (worse mo}$$

  - If K is too big, you get a **reliable estimate of a bad number**!



- **Solution**:
  - Rule of thumb: 70-30 or 80-20 split between train and validation

Error From VC Analysis vs Learn-Validation Approach

- In general:

$$E_{out}(h) = E_{in}(h) + \text{generalization error}$$

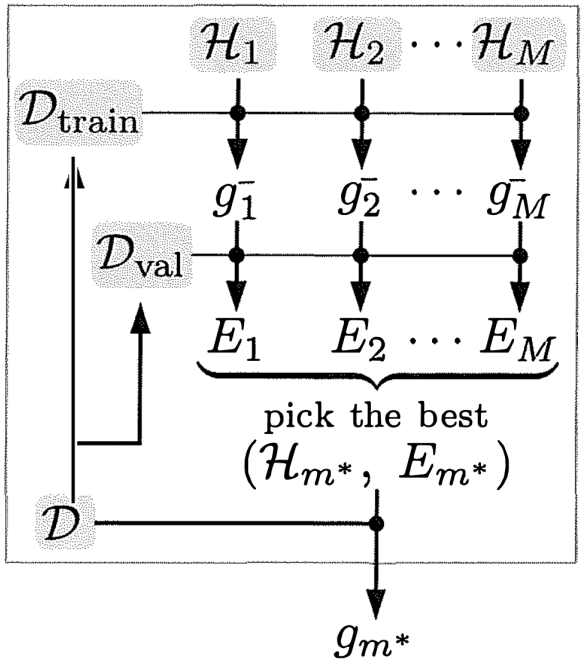
- **VC analysis**
  - Estimates generalization error as "overfit penalty" in terms of hypothesis set complexity
- **Learn-validation**
  - Estimates E_{out} directly by holding out data as validation set:

$$E_{val} \approx E_{out}$$

  - Use learn-validation approach at different points of the modeling flow:
    * For selecting hyperparameters (validation set)
    * To estimate final performance (test set)

Reusing Validation / Test Set for Training

- Any data used for learning (e.g., model training or model selection)
  - Is biased and optimistic
  - Cannot used to assess the learned model
- **Never use** D_{val} / D_{test} for training or D_{train} for evaluation
- After research is done, all available data can be used to learn
- **Algorithm**
  1. Train with $N - K$ points to learn g^-
  2. Use K points to compute $E_{val}[g^-]$ estimating $E_{out}[g^-]$
  3. Once the model form is finalized, use all N data points (including validation, test set) to re-train to get g
     - $E_{val}[g] < E_{val}[g^-]$ since g is learned on a larger data set than g^- and is thus better
  4. Deliver to customers:
     - Final hypothesis g



Learn-Validation Approach: Pros and Cons

- **Pros**
  - Estimate E_{out} using E_{val}
  - Simple to compute, no complexity from VC analysis
- **Cons**
  - Cannot use all data for learning and validation; need a compromise
  - Learned model and E_{val} depend on the split; different splits can give different results

- Learn-Validation Approach
  - Train / Test
  - ***Cross-Validation***

Cross-validation

- **Divide dataset** into K folds, each with $\frac{N}{K}$ samples
  - Each fold should reflect dataset statistics
  - E.g., stratified sampling
- **Do K iterations** $i = 1, ..., K$
- In i-th iteration
  - Train on all folds except i using $\frac{K-1}{K}N$ points and get $g^{(-i)}(\underline{\boldsymbol{x}})$
  - Validate on i-th fold using $\frac{N}{K}$ points to compute

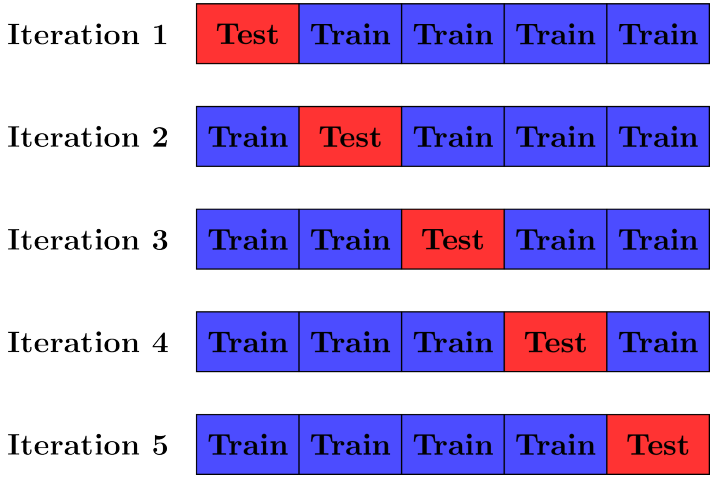
$$E_{val}^{(i)} = E_{val}[g^{(-i)}(\underline{\boldsymbol{x}})]$$

- **Average** K error rates and compute bounds:

$$E_{val} = \frac{1}{K}\sum_{i} E_{val}^{(i)}$$

Train / test validation

5x cross-validation



Cross-Validation: Pros and Cons

- **Pros**
  - Efficient data usage
    - All data used for learning and validation
  - Better estimate of E_{val} than separate training/validation sets
  - Folds can be stratified
- **Cons**
  - Computationally intensive: K learning phases
  - Dependency on fold selection
  - Errors E_{val} are not independent
    - Coupling through common training samples
    - Experimentally not completely correlated

Repeated Cross-Validation

- **Repeated Cross-Validation**
  - Cross-validation results depend on fold selection
  - To remove this dependency, repeat cross-validation multiple times (e.g., 10) and average results
  - Note that *"10x 10-fold cross-validation"* is different than *"1x 100-fold cross-validation"*

Leave-One-Out Cross-Validation

- **Leave-one-out** (LOO) cross-validation
  - There are N training sessions
  - Each session trains on $N-1$ points and validates on 1 point
  - Equivalent to "N-fold cross-validation" where N is the number of examples in the dataset
- **Train**
  - For i-th fold, $N-1$ samples for training $\implies g^{(-i)} \approx g$
- **Validate**
  - Estimate the validation error on a single point (bad):

$$E_{val}[g^{(-i)}] = e(g^{(-i)}(\underline{\boldsymbol{x}}_i), y_i) \not\approx E_{out}[g^{(-i)}]$$

  - Average E_{val} over the points as estimate of E_{out} (good):

$$E_{val} = \frac{1}{N} \sum_{i=1}^{K} E_{val}[g^{(-i)}]$$

Leave-One-Out Cross-Validation: Pros and Cons

- **Pros**
  - Max data used for training
  - Deterministic procedure
    - No dependency on fold selection
- **Cons**
  - High computational cost (as many learning phases as data points)
  - Cannot be stratified
  - Higher correlation between cross-validation estimates

Bootstrap

- **Algorithm**
  - Pick N samples with replacement from a data set with N instances to build training set
  - Pick the elements never chosen to build the test set ("out-of-bag" samples)
  - Training set contains 63.2% of all the samples, 36.8% in the test set
- **Pros**
  - Works for small data sets since it "expands" the data
- **Cons**
  - Not flexible
  - Smaller percentage of instances are used for training set than 10-fold cross validation

Bootstrap: Problem

- **Assumption**
  - Given $X \sim F$
  - Draw n IID samples X_i from F
- **Goal**:
  - Consider a sampling statistic $T = g(X_1, ..., X_n)$
    - E.g., mean, median, OLS coefficients, ...
  - You want to:
    - Estimate distribution of T, e.g., $F_T(x), f_T(x)$
    - Estimate a statistic of T, e.g., mean, std err
    - Construct confidence intervals, e.g., $\mu \pm \epsilon$
    - Calculate standard errors, e.g., $\sigma(\hat{T}_n)$
    - Hypothesis testing

Bootstrap Procedure: Algorithm

- Use observed data $X_1, ..., X_n$ to construct estimated population distribution $\hat{F}_n$
- Repeat B times:
  - Draw n samples with replacement from estimated population distribution $\hat{F}_n$
  - Sampling with replacement from X_i equals sampling from $\hat{F}_n$
  - Compute sample statistics from n samples: $T^{(i)} = g(X_1^{(i)}, ..., X_n^{(i)})$
- Use B samples of $T^{(i)}$ to estimate empirical distribution $\hat{T}$
- Compute statistics (e.g., confidence interval, standard error) of T from empirical distribution of $\hat{T}$

Bootstrap: Pros

- **Tremendously useful tool**
  - Fewer assumptions
    - No need for simplifying assumptions for closed formulas
    - Data doesn't need to be Gaussian
  - Generality
    - Applies to any sample statistics, even non-linear ones (e.g., median)
- **Math $\rightarrow$ simulation**
  - Bootstrap is a non-parametric method
  - Does not rely on large sample sizes, e.g., Central Limit Theorem (CLT) / Law Large Numbers (LLN)
  - Bootstrap frees data scientists from complex math, approximations, and asymptotics

Bootstrap: Example of Die Rolls

- **Problem**
  - Compute distribution of sum of rolling a die 50 times

$$Y = \sum_{i=1}^{50} X_i = g(X_1, ..., X_{50})$$

  - Sample statistics similar to sample mean, but can be anything
- **Solution**
  1. **By math**
     - If you know PMF of the die, compute distribution using mathematics
     - Theorem of lazy statistician for mean, variance
     - Compute PDF of Y by convolving PDFs
  2. **By sampling** (real or simulated)
     - Roll die 50 times
     - Compute sample statistic
     - Repeat procedure
     - Plot approximate distribution of Y
  3. If only 50 samples of die are known $\rightarrow$ **bootstrap**

Bootstrap of the Median: Pseudo-Code

```
def bootstrap_median(x, n_boot):
    # Compute n_boot sample statistics.
    median_boot = [0.0] * n_boot
    for i in range(n_boot):
        # Sample with replacement.
        x_star = sample with replacements from x
        # Compute median for bootstrapped samples.
        median_boot[i] = median(x_star)
    # Compute mean and std err from approximation of sample statistics.
    m_median = numpy.mean(median_boot)
    se_median = numpy.std(median_boot)
    return m_median, se_median
```

Bootstrap for variance of sample statistics: explanation

- Under the hypotheses of bootstrap:
  - $X \sim F$
  - n samples X_i IID from F
  - Statistic $T = g(X_1, ..., X_n)$
  - Compute $\mathbb{V}_F[T]$ (variance of sample statistics), where F is unknown
- **First approximation**
  - Only samples from F available
  - Approximate F with $\hat{F}$, as empirical CDF $\hat{F}$ converges to F

$$\mathbb{V}_F[T] \approx \mathbb{V}_{\hat{F}}[T]$$

  - Approximation
    - Not small
    - Depends on sample size and shape of F
- **Second approximation**
  - Exact $\hat{F}$ may lack closed formula for $\mathbb{V}_{\hat{F}}[T]$
  - Use LLN and simulation to approximate variance:

$$\mathbb{V}_{\hat{F}}[T] \approx v_{boot} = \frac{1}{B} \sum_i (T_i - \overline{T})^2$$